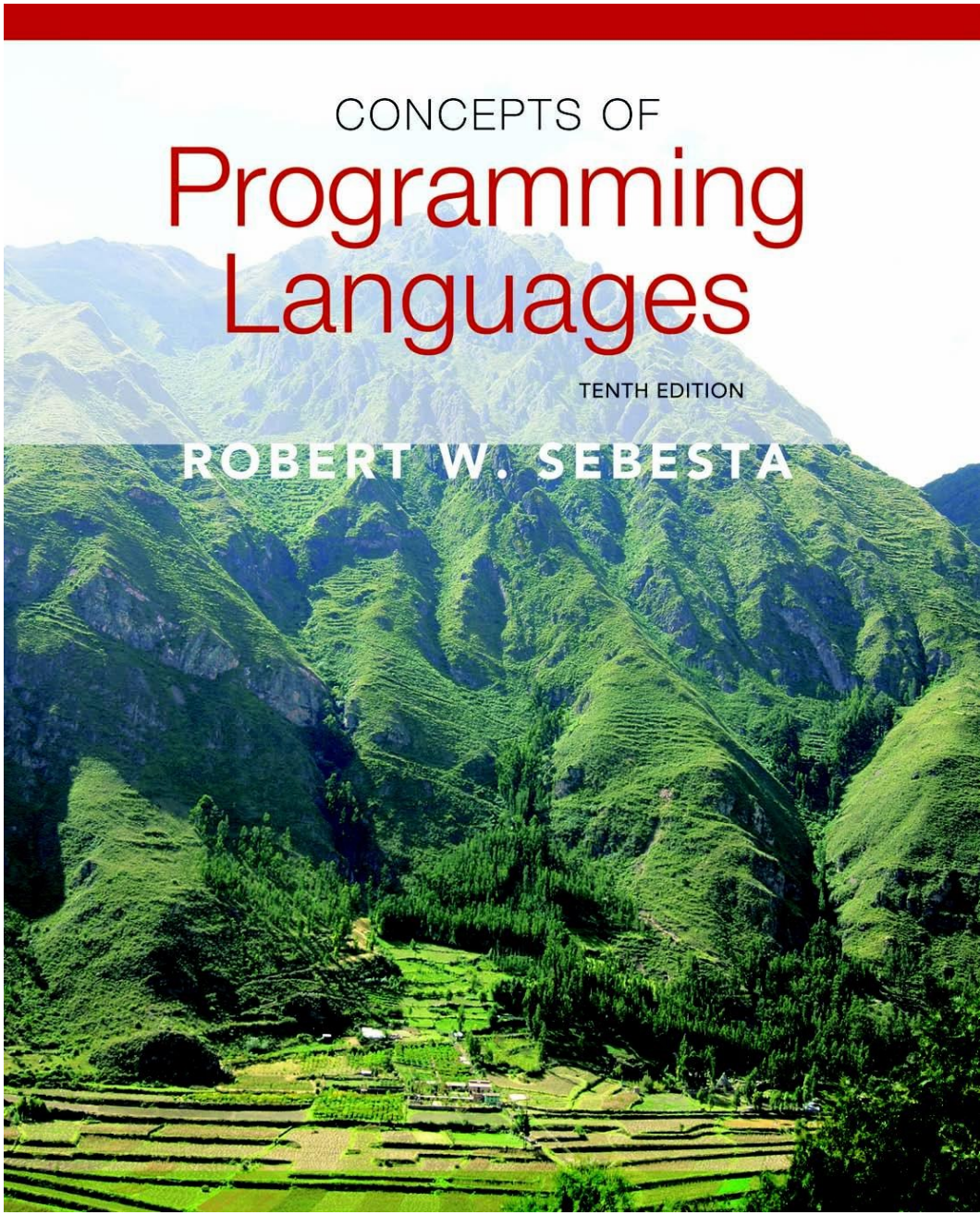
Chapter 11

Abstract Data Types and Encapsulation Concepts



Chapter 11 Topics

- The Concept of Abstraction
- Introduction to Data Abstraction
- Design Issues for Abstract Data Types
- Language Examples
- Parameterized Abstract Data Types
- Encapsulation Constructs
- Naming Encapsulations

Copyright © 2012 Addison-Wesley. All rights reserved.

The Concept of Abstraction

- An *abstraction* is a view or representation of an entity that includes only the most significant attributes
- The concept of abstraction is fundamental in programming (and computer science)
- Nearly all programming languages support process abstraction with subprograms
- Nearly all programming languages designed since 1980 support *data abstraction*

Copyright © 2012 Addison-Wesley. All rights reserved.

Introduction to Data Abstraction

- An *abstract data type* is a user-defined data type that satisfies the following two conditions:
  - The representation of objects of the type is hidden from the program units that use these objects, so the only operations possible are those provided in the type's definition
  - The declarations of the type and the protocols of the operations on objects of the type are contained in a single syntactic unit. Other program units are allowed to create variables of the defined type.

Copyright © 2012 Addison-Wesley. All rights reserved.

Advantages of Data Abstraction

- Advantages the first condition
  - Reliability--by hiding the data representations, user code cannot directly access objects of the type or depend on the representation, allowing the representation to be changed without affecting user code
  - Reduces the range of code and variables of which the programmer must be aware
  - Name conflicts are less likely
- Advantages of the second condition
  - Provides a method of program organization
  - Aids modifiability (everything associated with a data structure is together)
  - Separate compilation

Copyright © 2017 Pearson Education, Ltd. All rights reserved.

Example

Suppose an abstract data type is to be constructed for a stack that has the following abstract operations:

create(stack)	Creates and possibly initializes a stack object
destroy(stack)	Deallocates the storage for the stack
empty(stack)	A predicate (or Boolean) function that returns true if the specified stack is empty and false otherwise
push(stack, element)	Pushes the specified element on the specified stack
pop(stack)	Removes the top element from the specified stack
top(stack)	Returns a copy of the top element from the specified stack

A client of the stack type could have a code sequence

```
...
create(stk1);
push(stk1, color1);
push(stk1, color2);
temp = top(stk1);
...
```

Copyright © 2017 Pearson Education, Ltd. All rights reserved.

Language Requirements for ADTs

- A syntactic unit in which to encapsulate the type definition
- A method of making type names and subprogram headers visible to clients, while hiding actual definitions
- Some primitive operations must be built into the language processor

Copyright © 2012 Addison-Wesley. All rights reserved.

Language Examples: C++

- Based on C **`struct`** type
- The class is the encapsulation device
- A class is a type
- All the class instances of a class share a single copy of the member functions
- Each instance of a class has its own copy of the class data members
- Instances can be static, stack dynamic, or heap dynamic

Copyright © 2012 Addison-Wesley. All rights reserved.

Language Examples: C++ (continued)

- Information Hiding
  - *Private* clause for hidden entities
  - *Public* clause for interface entities

Copyright © 2012 Addison-Wesley. All rights reserved.

Language Examples: C++ (continued)

- Constructors:
  - Functions to initialize the data members of instances (they *do not* create the objects)
  - May also allocate storage if part of the object is heap-dynamic
  - Can include parameters to provide parameterization of the objects
  - Implicitly called when an instance is created
  - Can be explicitly called
  - Name is the same as the class name

Copyright © 2012 Addison-Wesley. All rights reserved.

Language Examples: C++ (continued)

- Destructors
  - Functions to cleanup after an instance is destroyed; usually just to reclaim heap storage
  - Implicitly called when the object's lifetime ends
  - Can be explicitly called
  - Name is the class name, preceded by a tilde (~)

Copyright © 2012 Addison-Wesley. All rights reserved.

An Example in C++

```
// Stack.h - the header file for the Stack class
#include <iostream.h>
class Stack {
private: //** These members are visible only to other members
  int *stackPtr;
  int maxLen;
  int topPtr;
public: //** These members are visible to clients
  Stack(); //** A constructor
  ~Stack(); //** A destructor
  void push(int);
  void pop();
  int top();
  int empty();
}
```

Copyright © 2012 Addison-Wesley. All rights reserved.

The code file for `Stack`

```
// Stack.cpp - the implementation file for the Stack class
#include <iostream.h>
#include "Stack.h"
using std::cout;
Stack::Stack() { //** A constructor
  stackPtr = new int [100];
  maxLen = 99;
  topPtr = -1;
}
Stack::~Stack() {delete [] stackPtr;}; //** A destructor
void Stack::push(int number) {
  if (topPtr == maxLen)
    cerr << "Error in push--stack is full\n";
  else stackPtr[++topPtr] = number;
}
...
```

Copyright © 2012 Addison-Wesley. All rights reserved.

Example program that uses the `Stack` ADT

```
void main() {
    int topOne;
    Stack stk; //** Create an instance of the Stack class
    stk.push(42);
    stk.push(17);
    topOne = stk.top();
    stk.pop();
    . . .
}
```

Copyright © 2012 Addison-Wesley. All rights reserved.

Language Examples: Java

- Similar to C++, except:
  - All user-defined types are classes
  - All objects are allocated from the heap and accessed through reference variables
  - Individual entities in classes have access control modifiers (private or public), rather than clauses
  - Java has a second scoping mechanism, package scope
    - All entities in all classes in a package that do not have access control modifiers are visible throughout the package

Copyright © 2012 Addison-Wesley. All rights reserved.

An Example in Java

```
class StackClass {
      private int [] stackRef;
      private int [] maxLen, topIndex;
      public StackClass() { // a constructor
            stackRef = new int [100];
            maxLen = 99;
            topPtr = -1;
      };
      public void push (int num) {…};
      public void pop () {…};
      public int top () {…};
      public boolean empty () {…};
}
```

Copyright © 2012 Addison-Wesley. All rights reserved.

An example class that uses `StackClass`

```
public class TstStack {
   public static void main(String[] args) {
      StackClass myStack = new StackClass();
      myStack.push(42);
      myStack.push(29);
      System.out.println("29 is: " + myStack.top());
      myStack.pop();
      System.out.println("42 is: " + myStack.top());
      myStack.pop();
      myStack.pop(); // Produces an error message
   }
}
```

Copyright © 2012 Addison-Wesley. All rights reserved.

Language Examples: C#

- Based on C++ and Java
- All class instances are heap dynamic like Java
- Default constructors are available for all classes
- Garbage collection is used for most heap objects, so destructors are rarely used
- **`structs`** are lightweight classes that do not support inheritance

Copyright © 2012 Addison-Wesley. All rights reserved.

Language Examples: C# (continued)

- Common solution to the need for access to data members: accessor methods (getter and setter)
- C# provides *properties* as a way of implementing getters and setters without requiring explicit method calls

Copyright © 2012 Addison-Wesley. All rights reserved.

C# Property Example

```
public class Weather {
   public int DegreeDays { //** DegreeDays is a property
      get {return degreeDays;}
      set {
        if (value < 0 || value > 30)
          Console.WriteLine(
              "Value is out of range: {0}", value);
        else degreeDays = value;}
   }
   private int degreeDays;
   ...
   }
...
Weather w = new Weather();
int degreeDaysToday, oldDegreeDays;
...
w.DegreeDays = degreeDaysToday;
...
oldDegreeDays = w.DegreeDays;
```

Copyright © 2012 Addison-Wesley. All rights reserved.

Parameterized Abstract Data Types

- Parameterized ADTs allow designing an ADT that can store any type elements – only an issue for static typed languages
- Also known as generic classes
- C++, Ada, Java 5.0, and C# 2005 provide support for parameterized ADTs

Copyright © 2012 Addison-Wesley. All rights reserved.

Parameterized ADTs in C++

- Classes can be somewhat generic by writing parameterized constructor functions

```
Stack (int size) {
  stk_ptr = new int [size];
  max_len = size - 1;
  top = -1;
};
```

A declaration of a stack object:

```
Stack stk(150);
```

Copyright © 2012 Addison-Wesley. All rights reserved.

Parameterized ADTs in C++ (continued)

- The stack element type can be parameterized by making the class a templated class

```
template <class Type>
class Stack {
  private:
    Type *stackPtr;
    const int maxLen;
    int topPtr;
  public:
    Stack() {  // Constructor for 100 elements
      stackPtr = new Type[100];
      maxLen = 99;
      topPtr = -1;
    }
    Stack(int size) {  // Constructor for a given number
      stackPtr = new Type[size];
      maxLen = size - 1;
      topSub = -1;
    }
  ...
}
```

  - Instantiation: `Stack<int> myIntStack;`

Copyright © 2012 Addison-Wesley. All rights reserved.

Parameterized Classes in Java 5.0

- Generic parameters must be classes
- Most common generic types are the collection types, such as `LinkedList` and `ArrayList`
- Eliminate the need to cast objects that are removed
- Eliminate the problem of having multiple types in a structure
- Users can define generic classes
- Generic user-defined classes cannot store primitives
- Indexing is not supported
- Example of the use of a predefined generic class:

```
ArrayList <Integer> myArray = new ArrayList <Integer> ();
myArray.add(0, 47);  // Put an element with subscript 0 in it
```

Copyright © 2012 Addison-Wesley. All rights reserved.

Parameterized Classes in Java 5.0 (continued)

```
import java.util.*;
public class Stack2<T> {
  private ArrayList<T> stackRef;
  private int maxLen;
  public Stack2() {
    stackRef = new ArrayList<T> ();
    maxLen = 99;
  }
  public void push(T newValue) {
    if (stackRef.size() == maxLen)
      System.out.println("Error in push - stack is full");
    else
      stackRef.add(newValue);
  ...
}
```

- Instantiation: `Stack2<String> myStack = new Stack2<String> ();`

Copyright © 2012 Addison-Wesley. All rights reserved.

Encapsulation Constructs

- Large programs have two special needs:
  - Some means of organization, other than simply division into subprograms
  - Some means of partial compilation (compilation units that are smaller than the whole program)
- Obvious solution: a grouping of subprograms that are logically related into a unit that can be separately compiled (compilation units)
- Such collections are called *encapsulation*

Copyright © 2012 Addison-Wesley. All rights reserved.

Encapsulation in C

- Files containing one or more subprograms can be independently compiled
- The interface is placed in a *header file*
- Problem: the linker does not check types between a header and associated implementation
- `#include` preprocessor specification – used to include header files in applications

Copyright © 2012 Addison-Wesley. All rights reserved.

Encapsulation in C++

- Can define header and code files, similar to those of C
- Or, classes can be used for encapsulation
  - The class is used as the interface (prototypes)
  - The member definitions are defined in a separate file
- *friend*s provide a way to grant access to private members of a class

Copyright © 2012 Addison-Wesley. All rights reserved.

Naming Encapsulations

- Large programs define many global names; need a way to divide into logical groupings
- A *naming encapsulation* is used to create a new scope for names
- C++ Namespaces
  - Can place each library in its own namespace and qualify names used outside with the namespace

Copyright © 2012 Addison-Wesley. All rights reserved.

C++ Namespaces

```
namespace myStackSpace {
 // Stack declarations
}
…
myStackSpace::topSub;
…
using myStackSpace::topSub;

using namespace myStackSpace;
p = topSub;
```

Copyright © 2012 Addison-Wesley. All rights reserved.

Naming Encapsulations (continued)

- Java Packages
  - Packages can contain more than one class definition; classes in a package are *partial* friends
  - Clients of a package can use fully qualified name or use the ***`import`*** declaration

Copyright © 2012 Addison-Wesley. All rights reserved.